---
title: "论文阅读：FedEditor - Efficient and Effective Federated Unlearning"
description: "Yuan 等 - 2025 - FedEditor: Efficient and Effective Federated Unlearning 论文个人分析"
slug: "fededitor-paper-reading"
date: 2026-02-13
categories:

- "笔记"
  tags:
- 联邦遗忘学习
- Federated Unlearning
- Federated Learning
- FedEditor
- 论文阅读
- 深度学习
  image:
  math: true
  draft: false

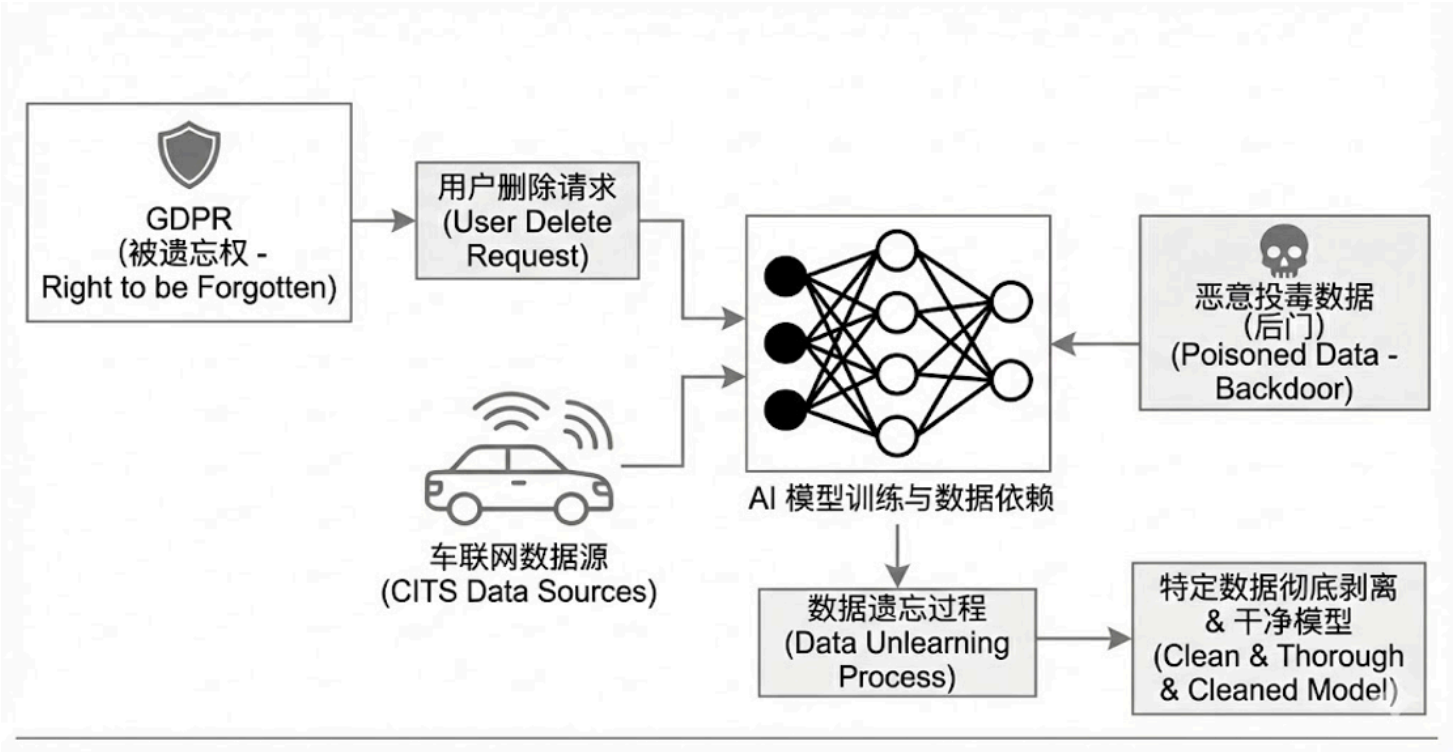
> **学术声明 / Academic Statement**
> 本文是个人学习笔记，仅用于学术交流和知识分享，非商业用途。
> 文中观点为作者个人理解和分析，不代表原论文作者立场。
> 如需引用原论文，请查阅官方出版版本。
> 版权归原论文作者和 IEEE 所有。

1. 论文基本信息

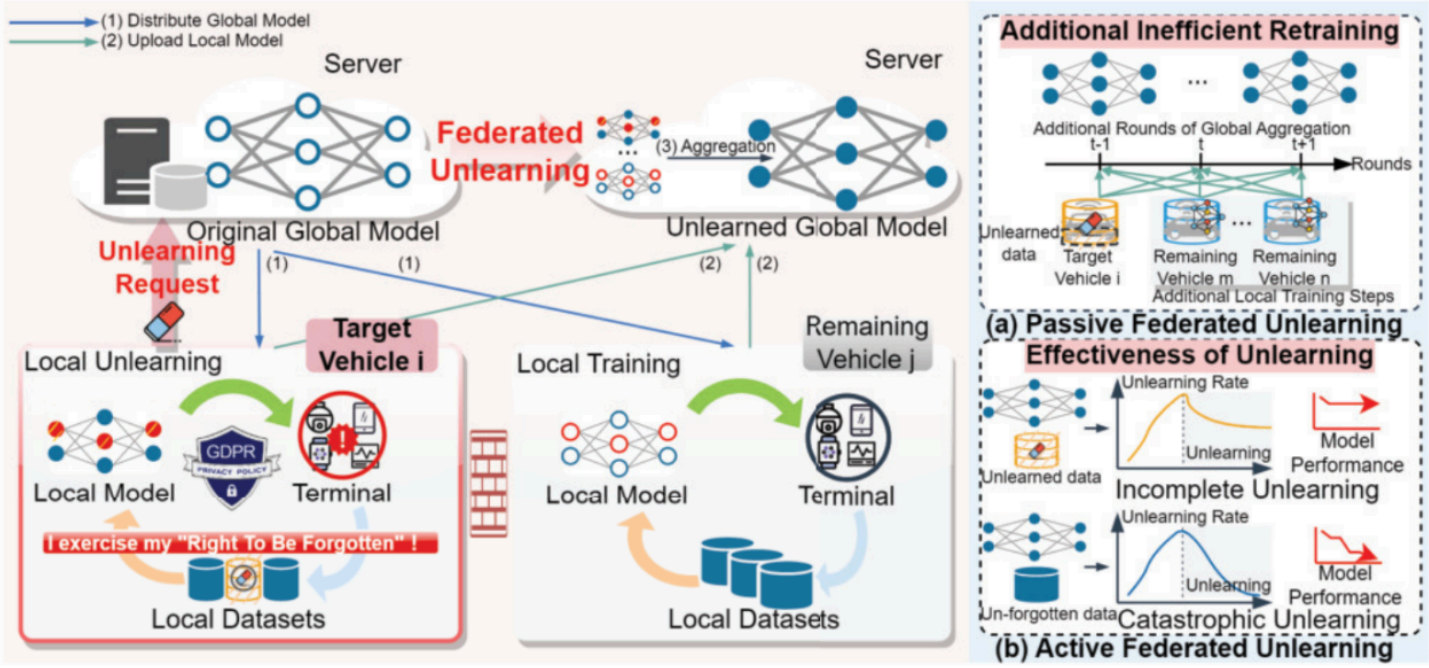
- **标题**：FedEditor: Efficient and Effective Federated Unlearning in Cooperative Intelligent Transportation Systems
- **作者**：Xiaohan Yuan, Jiqiang Liu, Bin Wang, Guorong Chen, Xiangrui Xu, Junyong Wang, Tao Li, Wei Wang
- **单位**：Beijing Jiaotong University, Haihe Lab of ITAI
- **发表期刊**：IEEE Transactions on Information Forensics and Security, Vol. 20, 2025
- **页码**：6560-6575

- **DOI**：10.1109/TIFS.2025.3583231
- **GitHub**：https://github.com/XXiaoY/Fededitor

2. 背景与痛点



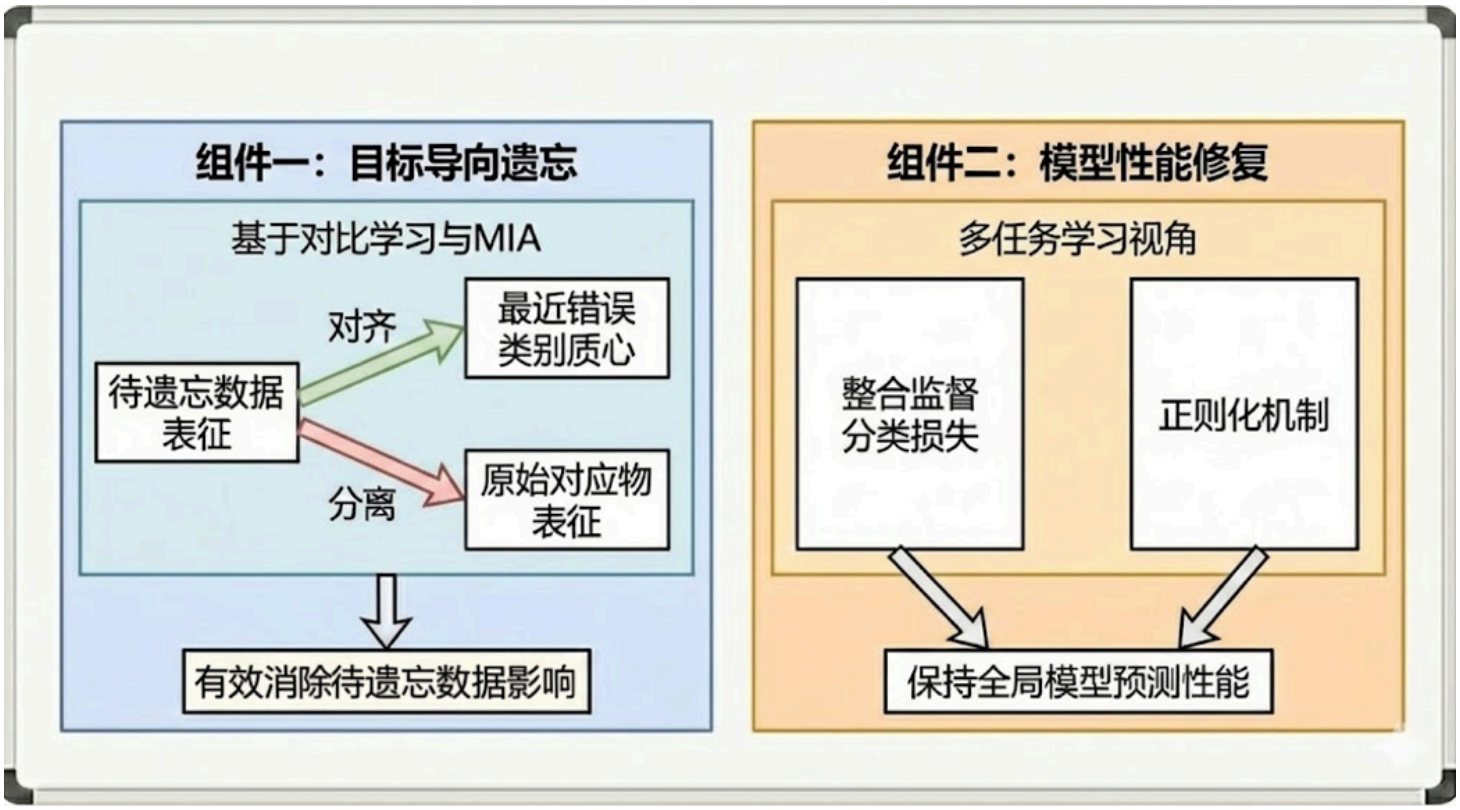
在车联网时代，AI 模型极其依赖海量数据的训练。但随之而来的是严峻的隐私挑战。
随着 GDPR 等法规的出台，‘被遗忘权’成了硬性指标。无论是用户要求删除隐私数据，还是我们发现某批数据被恶意投毒，系统都必须具备一种能力：从训练好的模型中，干净彻底地剥离掉特定数据的影响。

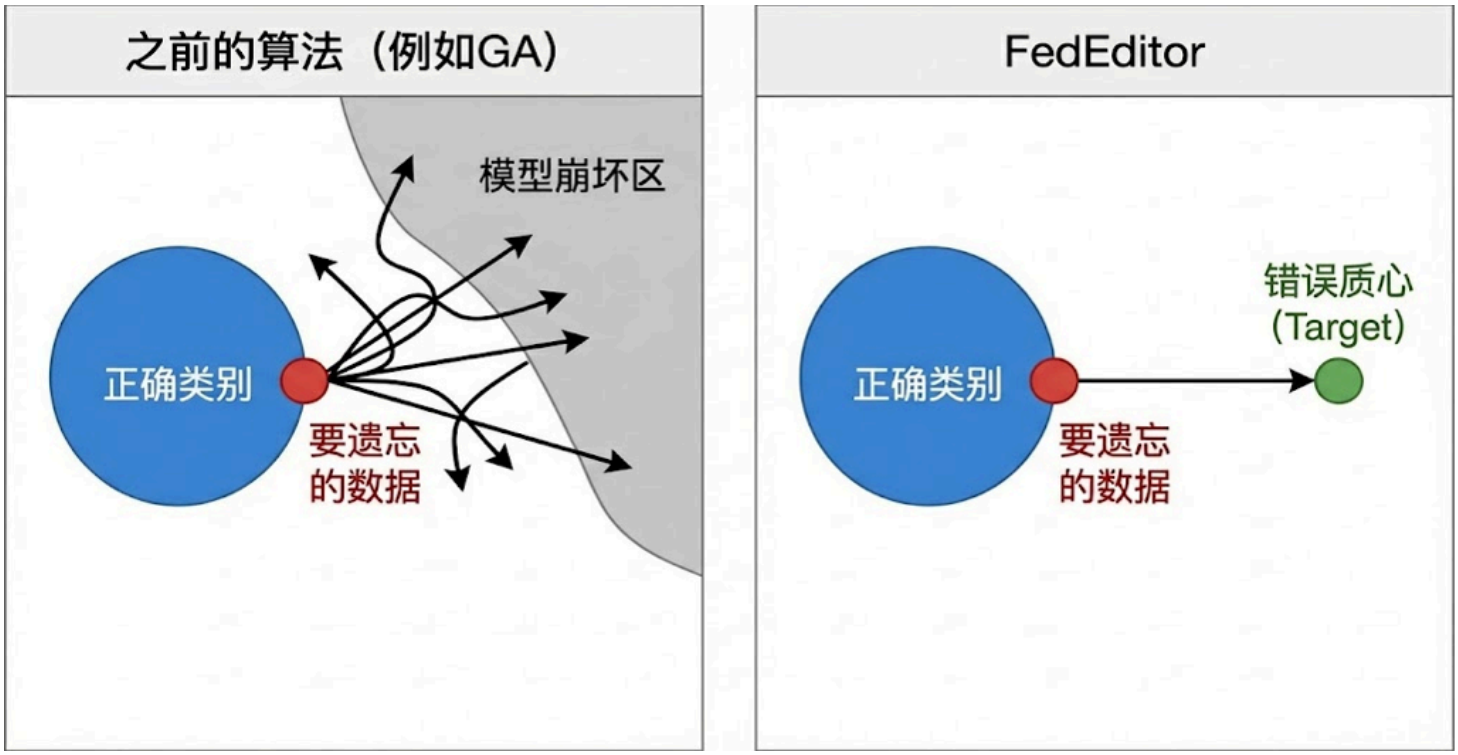


左边展示了联邦忘却学习在车联网中的标准流程，目标车辆在发起遗忘请求后，服务器将全局模型分发给所有客户端，目标车辆对模型进行修改，剩余车辆则继续进行本地训练，最后所有车辆把修改后的模型传回服务器，服务器对所有客户端的模型进行聚合后得到遗忘后的全局模型。

虽然流程已经定型，但是细节上却出现了各种问题，首先对于被动遗忘算法来说，他需要服务器带着所有客户端回退模型版本同时依靠剩余数据进行多轮重训，虽然他确实能达到较好的遗忘效果以及保持剩余数据预测的精确度，但是他需要所有客户端在线且耗时间很长，这对于车联网的应用场景是不可接受的，首先路况瞬息万变，长时间的训练会导致客户端无法及时获取最新的数据，不能及时应对变化的路况，其次车辆与服务器间的带宽无法支撑如此大规模的数据传输，最后，服务器无法确保能联系到所有车辆，如果车辆开上高速或开进山村，这都会导致服务器和车辆失联，所以被动联邦遗忘学习无法适用协作智能交通系统的场景。

而主动联邦遗忘算法，因为缺乏明确的优化目标，只能笼统的进行优化，这就可能会导致模型为达到遗忘的目标大幅修改参数，最终导致数据遗忘不彻底，或者是剩余数据灾难性遗忘的结果。为此有人提出对参数的修改范围进行界定，通过预设阈值的方法尝试使遗忘模型和原始模型对剩余数据的表现一致，但由于缺乏先验知识，没人清楚删除待遗忘数据后的模型参数长什么样，所以这个阈值难以设定。



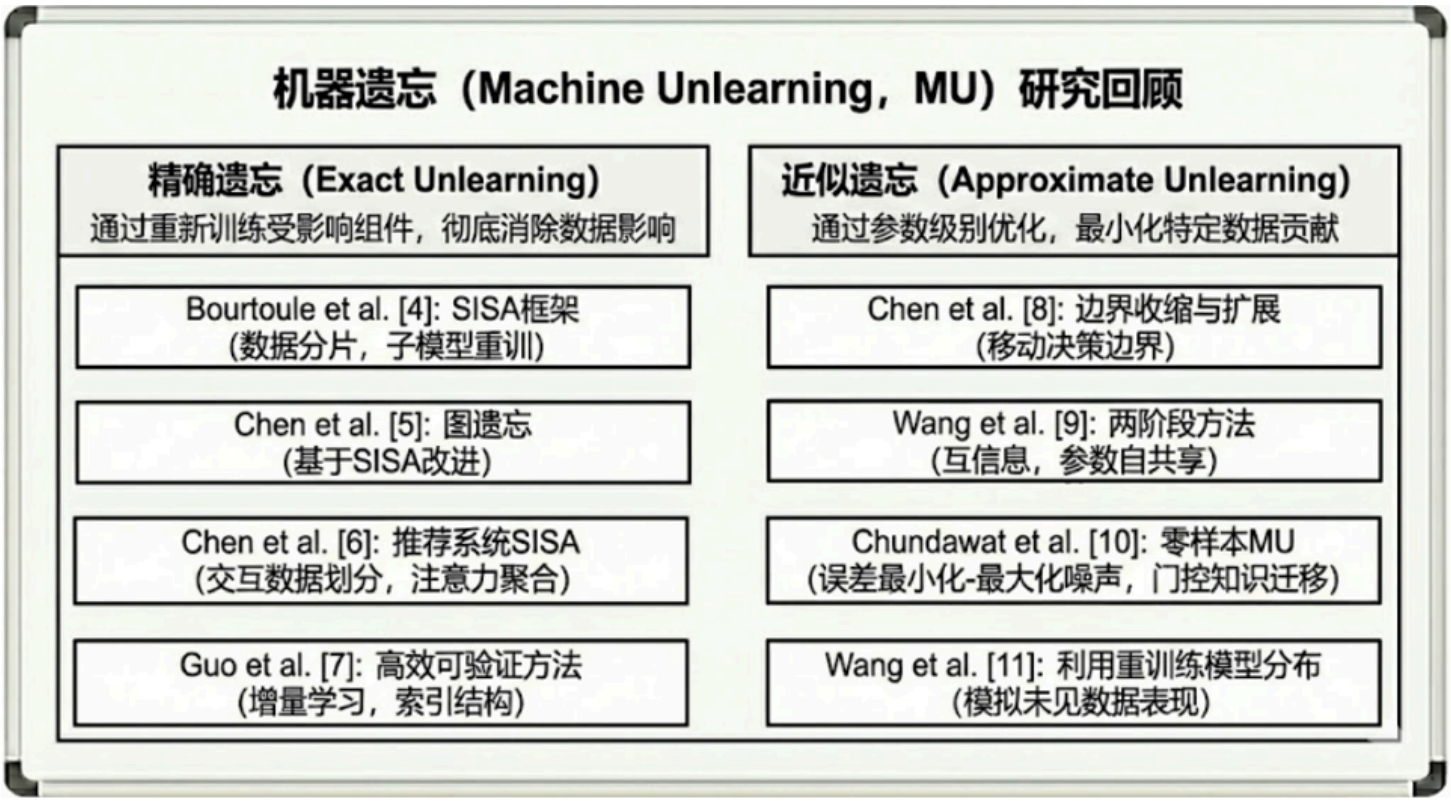


为此，我们提出了 FedEditor 算法，首先在定位上他是一个高效的主动遗忘算法，与被动重训不同，它允许车辆在本地主动进行修改，其次在方法上 FedEditor 创新性的提出了表征层面的局部遗忘策略

他包含两个核心部件，一个是目标导向遗忘，他通过引入错误质心来作为目标，解决了我们刚才提到的缺乏目标的问题，另一个是模型性能修复，他利用剩余数据和正则化约束保护模型的关键知识，防止灾难性遗忘

接下来我们将详细讲解算法细节

3. 算法核心思路

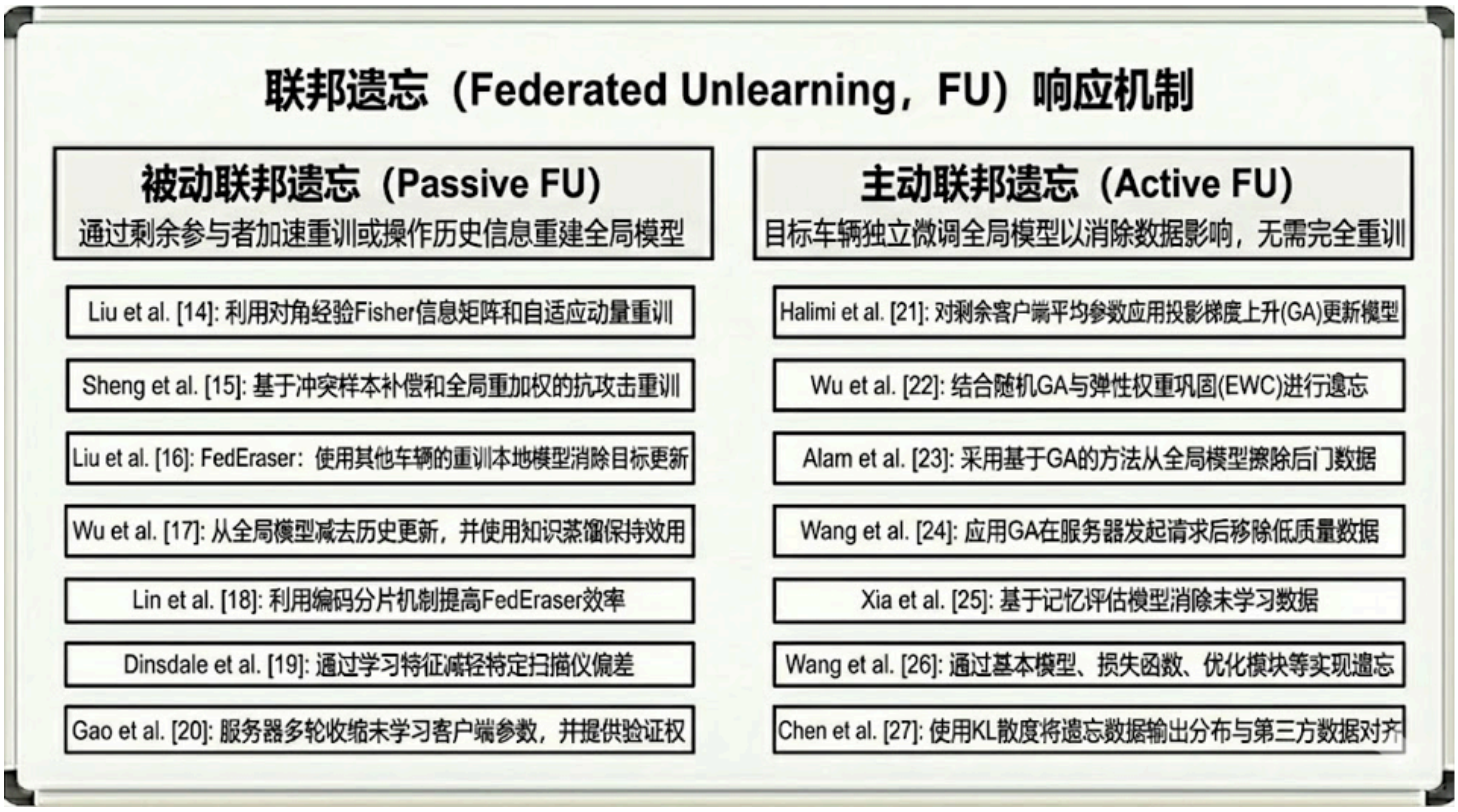


论文首先回顾了集中式场景下的传统机器遗忘方法。尽管这些方法均面向拥有完整数据的单体服务器设计，但其核心思路为联邦遗忘提供了重要参考，大致可分为两类：

精确遗忘的代表是 SISA 框架，其核心策略是**数据分片**：将训练数据预先切分为若干独立子集，当遗忘请求到来时，仅需对包含目标数据的子模型进行重训，而无需重训整个模型，从而显著降低了时间开销。

近似遗忘则放弃完全重训的追求，转而通过直接修改模型参数来近似达到遗忘效果。典型方法包括：**决策边界收缩与扩展**，针对分类器进行微调，通过调整决策边界来消除目标数据的影响；以及**参数分布对齐**，通过调整当前模型的参数分布，使其尽量逼近一个从未见过遗忘数据的理想模型。

然而，上述方法均基于集中式数据假设，无法直接迁移到数据分散、通信受限的车联网联邦学习场景，这也正是本文需要解决的核心问题。

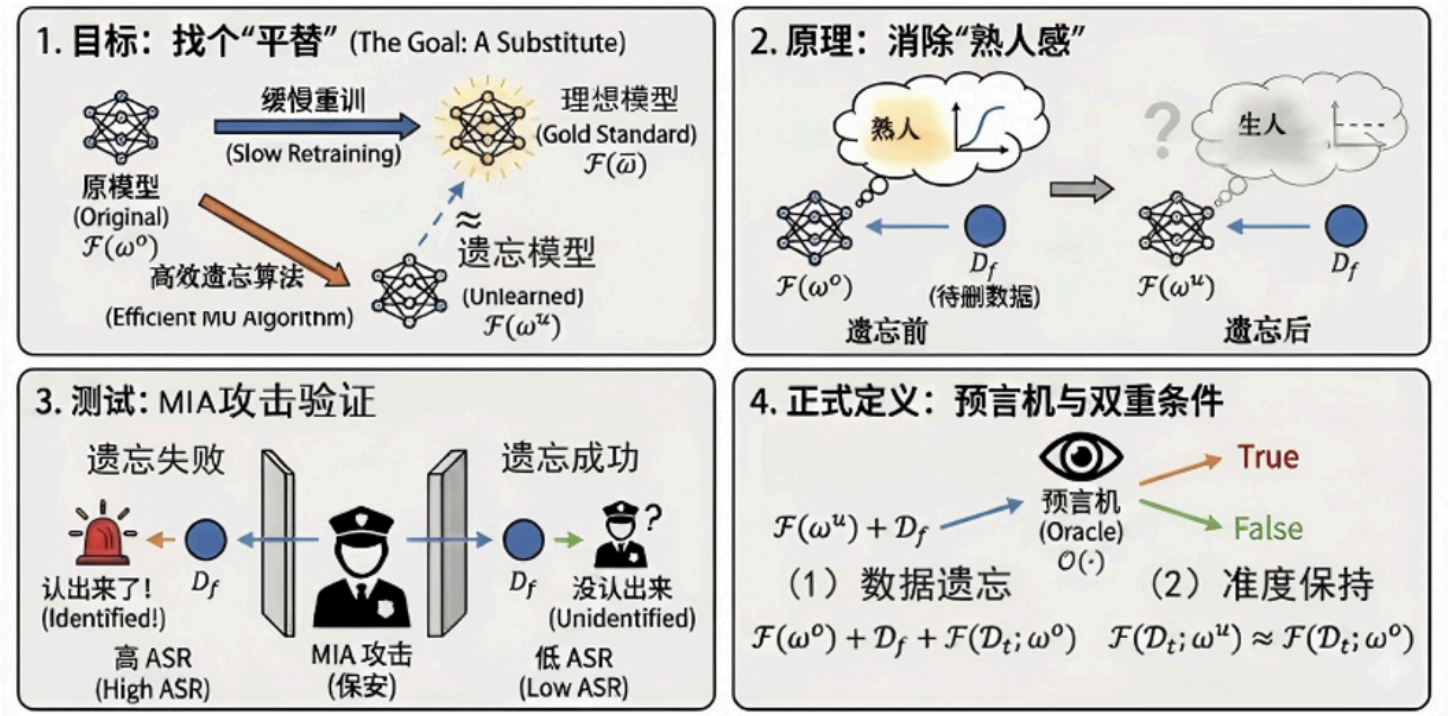


当前联邦遗忘算法可分为**被动联邦遗忘**与**主动联邦遗忘**两大类。

被动联邦遗忘依赖服务器协调所有客户端共同执行遗忘操作，主要方法有两种：其一是 **FedEraser**，它不再对模型进行完全重训，而是利用历史梯度更新记录进行参数回溯，再引导剩余客户端在当前基础上进行多轮校准训练，从而间接消除目标数据在历史更新中留下的影响；其二是文献 [17] 提出的贡献回滚方案，服务器持续记录每个客户端的参数贡献量，遗忘时直接撤销目标客户端的历史贡献，再对全局模型进行修复。然而，被动联邦遗忘对服务器的持续在线和大量通信开销有较强依赖，难以适应车联网的动态连接环境。

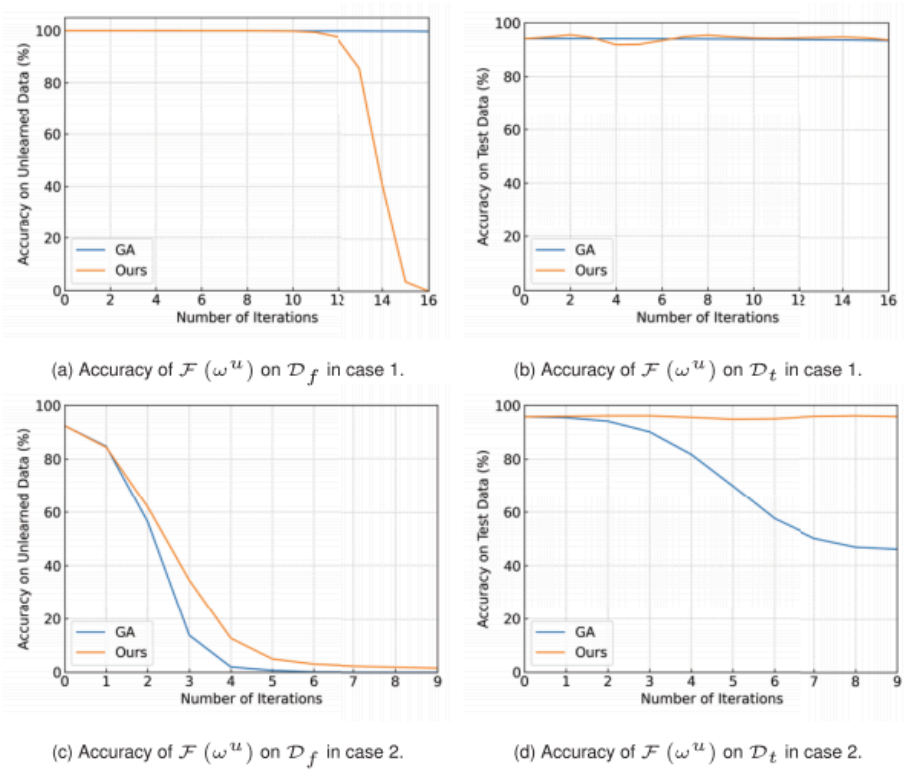
主动联邦遗忘则由客户端在本地独立完成遗忘操作，无需服务器深度介入。最直接的思路是**梯度上升（Gradient Ascent）**：既然模型通过梯度下降"记住"了训练数据，反向施加梯度上升便能让模型"忘记"目标数据。但纯粹的梯度上升副作用显著，容易在消除目标数据影响的同时破坏模型在剩余数据上的知识，因此后续工作引入了 **EWC（Elastic Weight Consolidation）** 正则化约束，对参数修改幅度加以限制，以保护关键权重不被过度扰动。此外，文献 [27] 提出的方法与本文的 FedEditor 有异曲同工之妙：该方法通过 **KL 散度**约束遗忘数据的输出分布，使当前模型的行为趋近于一个从未见过目标数据的理想模型。两者的关键区别在于作用层次——文献 [27] 的约束作用于**分类器的输出层**，而 FedEditor 则直接干预更底层的**特征提取器**，实现表征层面的精准遗忘。

通过对上述方法的系统梳理，FedEditor 的设计目标逐渐清晰：**遗忘要彻底**（目标数据的影响须被完全消除）、**知识要保留**（模型在剩余数据上的性能不能出现灾难性下降）、**效率要高**（在车联网这种实时性要求极高的场景下，遗忘过程必须足够轻量和快速）。



具体来说，我们需要通过高效的遗忘算法生成一个遗忘模型，使其在性能和行为上尽可能接近理想的“理想模型”(即完全移除目标数据后重新训练得到的模型)。其核心思想是通过特定手段，使遗忘后的模型在待遗忘数据上的表现与对非成员数据类似。从评估角度来看，是否成功遗忘的标准通常采用成员推理攻

击：训练一个判别模型来区分数据是否为成员。如果新模型能够有效误导该判别器，使其将待遗忘数据识别为非成员，则视为遗忘成功。同时，新的模型还需确保对其他数据的准确性不受影响，也就是说，其特征提取结果需与原模型高度一致，实现知识的完整保留。



(a) Accuracy of $\mathcal{F}(\omega^{\mathcal{U}})$ on $\mathcal{D}_f$ in case 1.

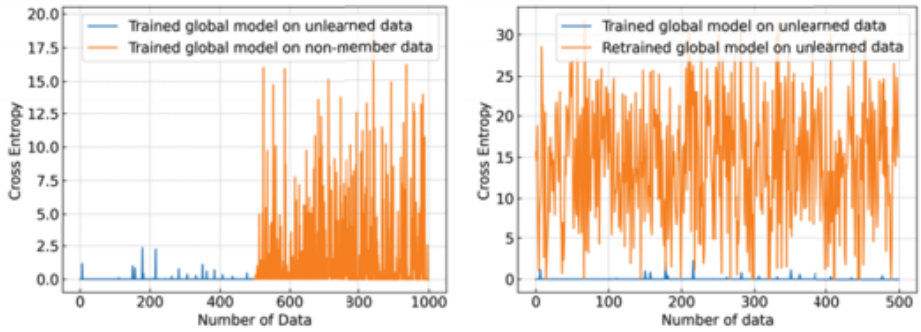
(b) Accuracy of $\mathcal{F}(\omega^{\mathcal{U}})$ on $\mathcal{D}_t$ in case 1.

(c) Accuracy of $\mathcal{F}(\omega^{\mathcal{U}})$ on $\mathcal{D}_f$ in case 2.

(d) Accuracy of $\mathcal{F}(\omega^{\mathcal{U}})$ on $\mathcal{D}_t$ in case 2.

Fig. 2. Accuracy of $\mathcal{F}(\omega^u)$ on $\mathcal{D}_f$ and $\mathcal{D}_t$.

然而，传统的被动联邦遗忘算法由于耗时过长难以满足实际需求，而梯度上升方法效果也有限——在模型过拟合时难以彻底遗忘目标数据，正常情况下又容易经过多轮训练后导致剩余知识的灾难性遗忘。面对这些局限，作者选择突破常规，创新性地提出了全新的解决思路。



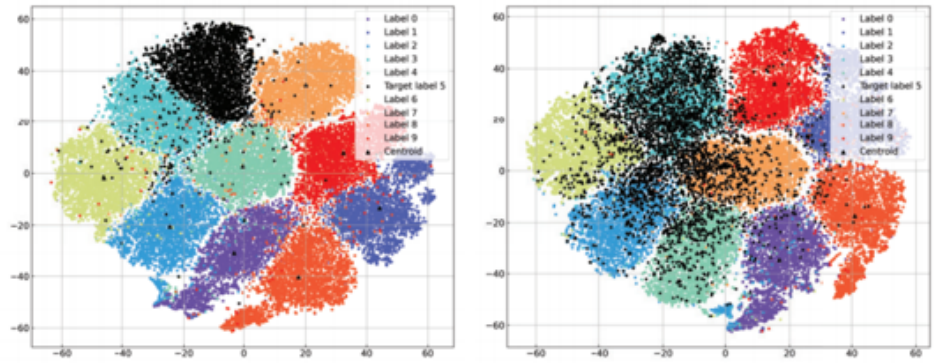
(a) Cross entropy of $\mathcal{F}(\omega^o)$ on $\mathcal{D}_f$ and $\mathcal{D}_t$.

(b) Cross entropy of $\mathcal{F}(\omega^o)$ and $\mathcal{F}(\bar{\omega})$ on $\mathcal{D}_f$.

Fig. 3. Cross entropy of $\mathcal{F}(\omega^o)$ and $\mathcal{F}(\bar{\omega})$ on different data.

前面提到，为了让训练后的模型对遗忘数据的“看法”与非成员数据一致，我们需要找到合适的衡量指标。作者通过定量分析发现，在原始模型中，待遗忘（成员）数据的交叉熵一般较小且波动较小，说明模型对这些数据有高度自信；而对于非成员数据，交叉熵则普遍较大且波动很大，反映出模型缺乏信

心。在“理想模型”——即已经完全移除目标数据并重新训练得到的模型——中，待遗忘数据表现出与非成员数据一致的高交叉熵和不稳定性，表明这些数据已被模型“遗忘”。因此，如果希望模型把待遗忘数据当做“陌生人”，核心目标就是显著提升其交叉熵，使其表现接近非成员数据。



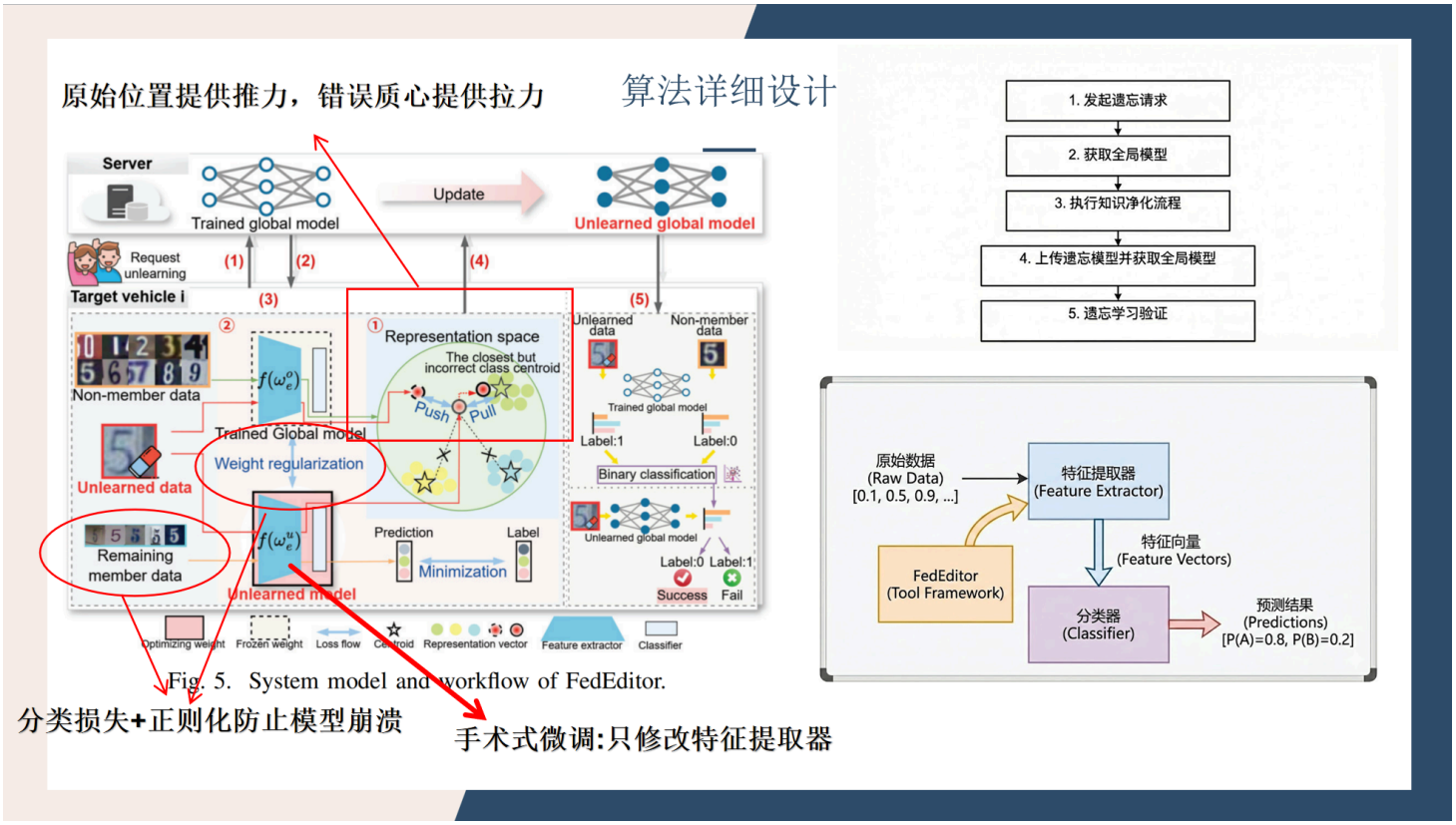
(a) $\mathcal{D}_f$ and $\mathcal{D}_r$ of $\mathcal{F}(\omega^O)$.　　(b) $\mathcal{D}_f$ and $\mathcal{D}_r$ of $\mathcal{F}(\bar{\omega})$.

Fig. 4. t-SNE visualizations of representation distributions.

除了定量指标之外，表征分布的可视化（如图 4）也进一步揭示了特征层的变化：原始模型下，成员数据的特征分布紧密且边界清晰，显示出模型强烈的区分能力；而在理想模型中，遗忘数据的特征分布变得分散模糊，完全融入背景，与成员数据融为一体，标志着模型已不再识别这些数据。从本质上讲，让待遗忘数据变成“陌生人”的关键，就是打破其原本紧密的聚类结构，使其在特征空间内变得杂乱、无序。

那么，如何高效且精确地打散这些特征，使遗忘彻底生效？接下来我们将详细介绍 FedEditor 的核心算法设计。

4. 核心方法详解

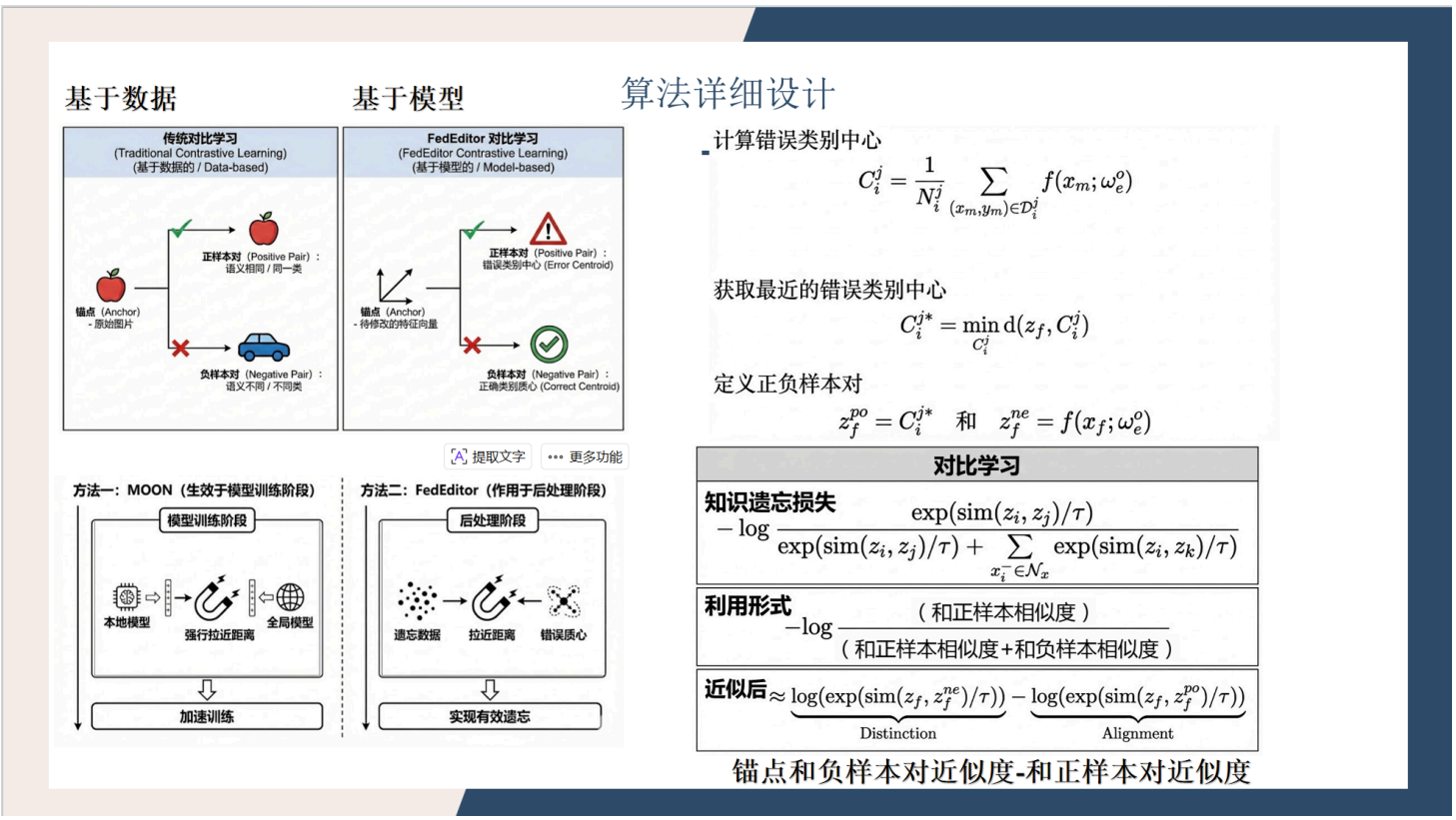


在论文中，FedEditor 的算法流程如图所示：当某个客户端发起遗忘请求后，先从服务器获取全局模型，随后仅针对特征提取器进行精细、局部的修改，实现主动遗忘。这一创新点区别于传统的联邦遗忘方法。与以往依赖梯度上升直接调整全局参数（如文献 11）或修改输出层（如文献 27）的做法不同，FedEditor 选择“手术式”地干预特征提取器，将遗忘作用精确聚焦于表征层。通过调整特征提取器的参数，打破原本紧密聚类的成员特征分布，使其变得分散、无序，从而逼近“理想模型”的特性。

可以类比为：特征提取器如同“眼睛”，分类器如同“大脑”。当眼睛准确地将“5”对应的特征传给大脑时，大脑很容易识别为数字 5。但如果改变了“眼睛”的参数，使其输出的特征与原标签弱相关，大脑也难以做出准确判断。这就是 FedEditor 遗忘的基本思想。

具体操作流程如下：首先利用非成员数据，通过原始模型的特征提取器计算每个错误类别的特征中心（绿色圆圈和星星，星星代表各类别均值中心）；再用待遗忘数据，通过同样的特征提取层获得其初始特征向量（左侧红点）。对于每一个待遗忘样本，都会选择其最近的错误类别中心作为目标，通过优化使该样本经过修改后的特征提取器输出从原始位置逐步“推离”并朝该错误类别中心“靠拢”。在这一过程中，原始特征位置提供动力，目标位置产生牵引，参数变化通过权重正则化加以约束，分类损失保障剩余数据的预测准确性。最终，原本聚集的待遗忘样本的特征会因为各自最近的错误类别中心不同而被拉扯到不同方向，特征分布整体被打散，原标签与特征不再高度对应，分类器对它们的预测置信度降低、交叉熵显著升高，实现模型对其“陌生化”——使成员推理攻击也难以分辨，达到真正的遗忘效果。

> 注：为什么要用非成员数据来计算错误类别中心，而不是直接用训练数据呢？以“遗忘数字 5 的图片”为例，如果我们用训练集中“7”这一类别来作为错误类别中心，看似无害，但实际上，成员数据（即参与过训练的数据）与模型的拟合高度相关。如果用成员数据计算错误类别中心，待遗忘样本在特征空间被引导靠近这些点时，会不可避免地染上成员数据的特征分布。由于模型对成员数据本就具有高置信度，此时即便模型从“高置信度的 5”识别为“高置信度的 7”，置信度依然很高，反而让攻击者依旧容易通过成员推理攻击判断该样本参与过训练——即使预测标签错了，只要置信度高异常就仍有泄漏风险。因此，选取非成员数据（未被模型训练过的数据）计算错误类别中心，可以最大化减少遗忘过程中成员特征的“染色”，避免模型对这些点形成异常自信，从而实现更彻底的遗忘和更强的抗推理攻击能力。
> 至于“非成员数据”具体该如何选择，其实通常直接采用训练集对应的测试集即可——即便没有严格对应的测试集，选择其他相似分布的测试数据也完全可行。实验证明，只要数据未被模型训练过，都可以作为合格的非成员数据用于错误类别中心的计算。



前文提到，原始特征向量为推力，错误类别中心为拉力，这种说法容易流于抽象。那么，FedEditor 的遗忘机制究竟是如何在算法层面实现的呢？这里，作者提出了基于模型的对比学习（contrastive learning）的创新思路。与传统对比学习仅对不同增强版本的同类样本进行“拉近正样本、推远负样本”不同，MOON 算法的突破在于将对比对象从数据维度扩展到了模型维度，通过让本地模型与全局模型间的特征距离拉近，提升联邦训练的稳定性。

FedEditor 则在此基础上进一步创新，对正负样本对的定义做了巧妙翻转：正样本对设定为“错误类别中心”，负样本对则为“原始正确类别的特征中心”。在优化过程中，每一个待遗忘样本被有意拉近至与其标

签错误的类别中心，同时远离其原本所属类别中心。这一机制促使成员数据在特征空间内失去原有紧密聚类的结构，分布变得分散、模糊，从而达到遗忘目标并有效削弱成员推理攻击的能力。

相关的计算流程如下：首先，遍历所有非成员数据，计算得到各类别的错误类别中心。然后，对于每一个待遗忘样本，选择其最近的错误类别中心作为“正样本对”，并将原始正确类别的特征中心作为“负样本对”。在定义损失函数时，采用对比学习的经典结构：优化目标是增强锚点（待遗忘样本特征）与新目标（最近错误类别中心）之间的相似度，同时削弱与原类别中心的相似度。具体而言，分子部分衡量锚点与新目标的相似性，分母部分则综合了与新目标和旧目标的相似性。通过最小化该损失函数，可以推动特征提取器更好地将待遗忘数据的特征远离原类别，同时向错误类别靠拢，有效打散原有的聚类特征，从而实现遗忘目的。

算法详细设计

模型性能修复组件

记忆保留损失函数

$$\ell_r = -\sum y_{r,j} \log(p_{r,j})$$

偏差抵抗损失函数

$$\ell_d = d(\boldsymbol{w}^u, \boldsymbol{w}^o) = \frac{1}{2}\|\boldsymbol{w}^u - \boldsymbol{w}^o\|_2^2$$

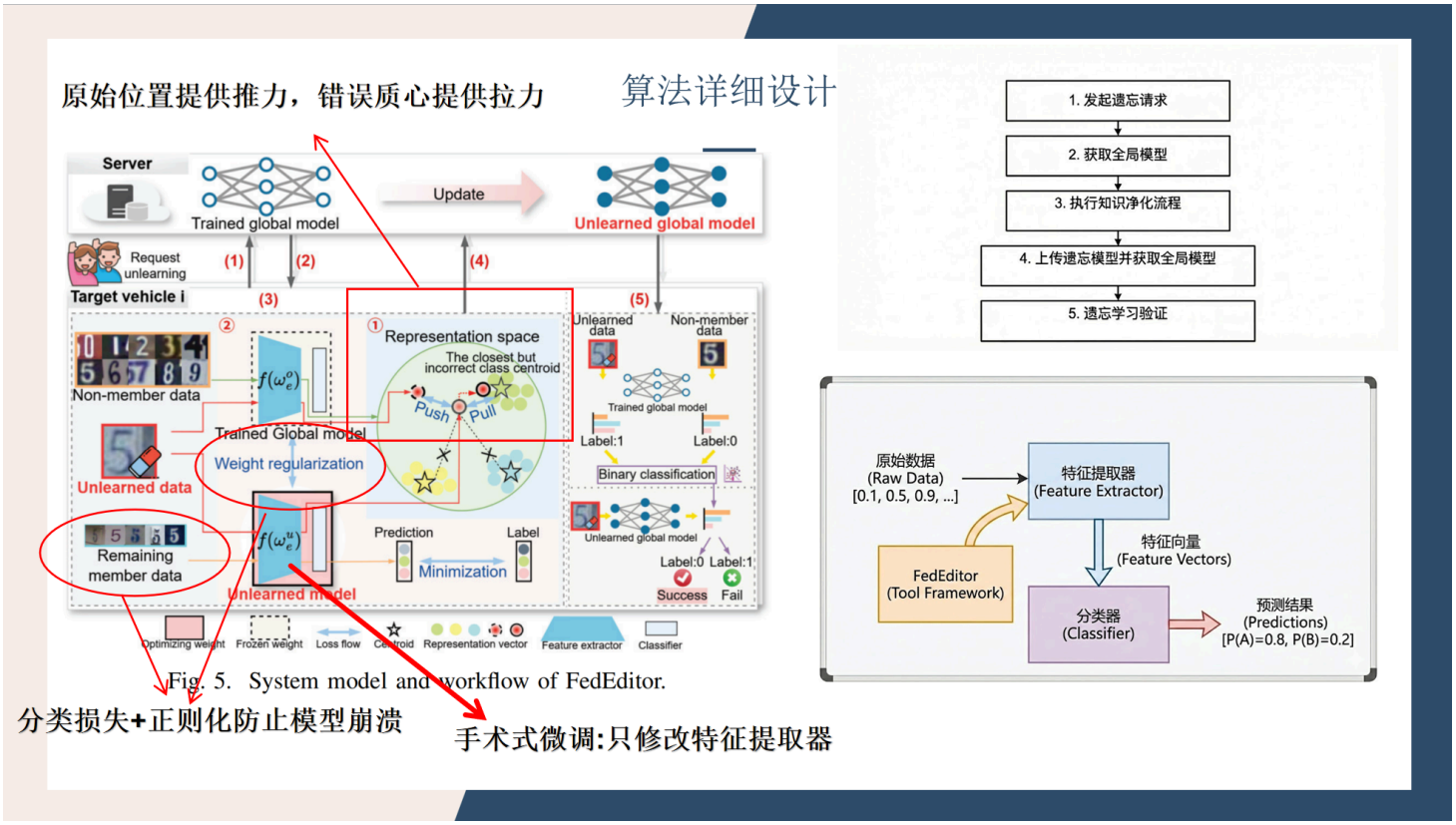
$$Total\ Loss = \underbrace{\alpha \cdot l_u}_{\text{往错误方向拉}} + \underbrace{\beta \cdot l_r}_{\text{保持正确数据准确}} + \underbrace{\gamma \cdot l_d}_{\text{不让参数跑太远}}$$

l_u：知识遗忘损失（Knowledge Unlearning Loss） 提供动力

l_r：记忆保留损失（Memory Preservation Loss） 保护精度

l_d：偏差抵抗损失（Deviation Resistance Loss） 保护结构

与此同时，模型修复模块中的两个关键损失函数在持续发挥作用：其一，记忆保留损失（即交叉熵损失）确保模型对剩余数据的预测准确性不受遗忘过程影响；其二，偏差抵抗损失（即权重正则化）则限制参数变化幅度，防止模型结构发生较大偏移。这三项损失（加上遗忘损失）通过加权求和，共同组成算法的全局损失函数，有效引导模型实现精准、可控的参数调整。



回到整体流程，在完成局部模型修改后，客户端将新模型上传到服务器，由服务器进行聚合。随后，为评估遗忘效果，会基于原始模型输出，用成员数据和非成员数据训练一个二分类器（成员推理攻击器），再利用该分类器检测新模型对待遗忘数据的判别效果。如果分类器在新模型上难以分辨成员与非成员数据，则判定遗忘成功。

5. 实验设计与结果

实验结果证明

问题一：FedEditor相较于被动FU方法的效率优势

问题二：FedEditor的遗忘能力

问题三：通过FedEditor保持全局模型预测性能的稳定性

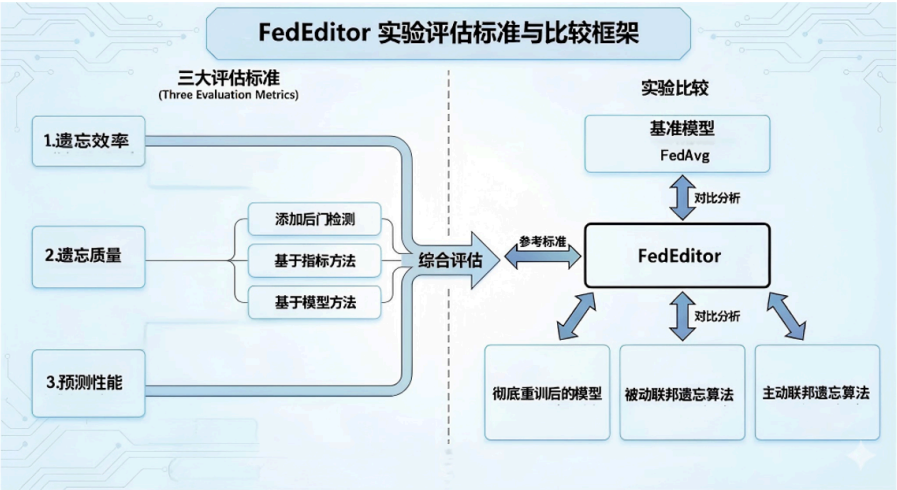
问题四：FedEditor的适用性

算法设计完成后，进入实验评估阶段，以全面验证所提方法的有效性与优越性。为此，作者围绕算法性能提出了四个核心问题，后续将结合具体实验数据一一解答。

实验结果证明

TABLE II
DATASETS AND MODELS

Datasets	Train instances	Test instances	Features	Classes	Models
FMNIST	60000	10000	28×28	10	LeNet5
CIFAR10	50000	10000	32×32	10	ResNet18
SVHN	73257	26032	32×32	10	ResNet18
CIFAR100	50000	10000	32×32	100	ResNet34
GTSRB	39209	12630	64×64	43	ResNet18
LOAN	361703	90430	91	9	MLP
BAIOT	440000	110000	115	11	CNN



实验部分，作者选用了五个经典图像数据集（FMNIST、CIFAR10、SVHN、CIFAR100、GTSRB）和两个表格数据集（LOAN 和 BAIOT）来全面验证方法的有效性。其中，FMNIST（Fashion-MNIST）用于服饰图像分类，CIFAR10 和 CIFAR100 用于通用物体识别，SVHN 主要针对街景数字识别，GTSRB 则用于交通标志分类；LOAN 和 BAIOT 数据集分别涉及贷款欺诈检测和物联网设备异常检测。实验评价指标包括遗忘效率、遗忘质量和预测性能三大类标准。在遗忘质量评估方面，作者同时采用了后门检测、基于指标的分析和基于模型的评测三种方法以确保结论的客观全面。对比方案则涵盖了基于 FedAvg 训练的原始模型、彻底重训练得到的理想模型，以及多个主流被动和主动联邦遗忘算法。

实验结果证明

遗忘效率

速度远超被动算法，且准确率和稳定性碾压其他主动算法。

TABLE III

COMPARISONS ON UNLEARNING EFFICIENCY, MODEL PREDICTIVE PERFORMANCE, AND UNLEARNING EFFECT ACROSS IMAGE DATASETS

Metrics	FedAvg	Re_train	Passive FU			Active FU					
			Eraser	Rap_train	Con_train	Ram_lab	GA	Ng_ewc	In_tea	PGD	Ours
FMNIST-LeNet											
Time(s)↓	991.85	730.92	139.03	668.16	14.76	3.44	3.77	39.63	**3.20**	3.93	5.79
Acc. on test data↑	91.22	91.32	85.37	87.62	91.26	80.46	70.04	78.26	73.42	82.07	**87.34**
Acc. on unlearned data↓	99.92	0.50	0.08	0.17	99.91	0.75	**0.00**	2.25	2.08	6.00	2.58
JSD to Re_train↓	0.0057	-	0.0001	0.0001	0.0058	**0.0001**	0.0005	0.0004	0.0003	0.0004	**0.0001**
AD to Re_train↓	1.3675	-	0.1914	0.1331	1.3673	**0.1311**	0.3605	0.3871	0.2945	0.3955	0.1665
ASR of MIA_metric→ 50.00	81.97	50.00	50.00	50.00	82.04	**50.00**	**50.00**	**50.00**	52.51	**50.00**	**50.00**
ASR of MIA_model→ 50.00	99.67	50.54	50.00	50.75	98.38	**49.96**	52.21	53.50	44.17	56.96	50.63
CIFAR10-ResNet18											
Time(s)↓	2089.48	1469.65	869.47	2471.21	31.48	8.83	7.78	181.87	19.04	**7.12**	19.80
Acc. on test data↑	66.64	68.90	55.69	68.34	67.08	54.28	63.78	64.03	59.35	54.54	**66.02**
Acc. on unlearned data↓	98.60	2.80	2.90	4.00	77.40	4.90	5.10	4.70	6.00	**3.60**	3.70
JSD to Re_train↓	0.0053	-	0.0009	0.0007	0.0031	0.0035	0.0013	0.0013	0.0041	**0.0011**	0.0012
AD to Re_train↓	1.3208	-	0.5298	0.4223	1.0602	0.9655	0.4519	**0.4453**	1.1530	0.5415	0.4806
ASR of MIA_metric→ 50.00	75.66	50.00	50.00	50.00	61.09	50.46	**50.00**	**50.00**	51.88	50.06	50.59
ASR of MIA_model→ 50.00	98.80	50.10	50.35	50.05	52.15	45.40	39.05	39.90	45.35	54.15	**48.95**
SVHN-ResNet18											
Time(s)↓	4161.02	2726.48	1054.01	4131.24	37.77	11.09	**10.65**	206.73	16.50	14.11	13.24
Acc. on test data↑	89.81	91.02	88.16	90.16	89.66	43.29	77.02	81.25	79.21	74.20	**88.66**
Acc. on unlearned data↓	96.93	1.02	1.36	1.50	7.58	9.56	**0.00**	0.61	1.02	2.11	3.14
JSD to Re_train↓	0.0045	-	0.0001	0.0001	0.0019	0.0035	0.0003	**0.0002**	0.0020	0.0007	**0.0002**
AD to Re_train↓	1.3465	-	0.1418	0.1091	0.7891	1.1602	0.1564	**0.1263**	0.8051	0.3894	0.1399
ASR of MIA_metric→ 50.00	58.28	50.00	50.00	50.00	50.07	**50.00**	**50.00**	**50.00**	**50.00**	**50.00**	50.02
ASR of MIA_model→ 50.00	96.07	50.34	51.57	49.93	48.87	49.69	**50.00**	50.44	49.52	50.17	49.69
CIFAR100-ResNet34											
Time(s)↓	3193.76	2220.50	868.36	3682.60	22.13	**4.74**	4.77	178.48	9.34	7.17	11.33
Acc. on test data↑	42.84	45.92	37.03	47.98	41.13	31.95	35.70	38.85	32.82	17.90	**45.90**
Acc. on unlearned data↓	98.70	3.30	1.90	3.40	76.10	4.00	0.60	**0.00**	5.60	44.30	0.80
JSD to Re_train↓	0.0052	-	0.0016	0.0014	0.0030	0.0031	0.0027	0.0026	0.0031	0.0036	**0.0021**
AD to Re_train↓	1.2841	-	0.6960	0.6507	1.0359	0.9273	0.7871	0.7553	0.9377	1.0244	**0.7048**
ASR of MIA_metric→ 50.00	84.66	50.00	50.00	50.00	68.49	**50.00**	**50.00**	**50.00**	53.49	63.97	51.16
ASR of MIA_model→ 50.00	97.60	50.00	50.10	49.55	48.75	49.00	52.15	52.20	48.80	51.20	**50.20**
GTSRB-ResNet18											
Time(s)↓	1366.17	781.99	523.14	1144.07	13.57	9.25	8.24	84.76	12.45	**5.48**	22.58
Acc. on test data↑	58.54	53.41	20.48	66.61	54.89	50.26	55.03	55.75	30.64	38.01	**57.51**
Acc. on unlearned data↓	97.93	5.07	3.75	10.71	98.12	**0.19**	4.13	1.69	34.21	2.63	2.25
JSD to Re_train↓	0.0062	-	0.0029	0.0005	0.0080	0.0050	0.0014	0.0017	0.0060	0.0027	**0.0006**
AD to Re_train↓	1.1797	-	0.7650	0.3814	1.1916	0.8309	0.4961	0.4780	0.8739	0.6910	**0.4629**
ASR of MIA_metric→ 50.00	84.32	50.00	50.00	50.00	90.85	52.98	**50.00**	**50.00**	75.15	**50.00**	50.08
ASR of MIA_model→ 50.00	99.53	50.28	49.71	48.21	93.14	49.53	53.28	55.26	49.71	50.84	**49.91**

输出分布差异

模型结构差异

问题一：FedEditor相较于被动FU方法的效率优势

在训练效率方面，被动遗忘算法虽然通常能够获得更高的准确率，但其训练过程十分耗时。比如 Eraser 的整体流程复杂，Rap_train 需要计算二阶导数矩阵，Con_train 则需用剩余数据重新训练多轮，因此耗费的时间成本较高。相比之下，主动遗忘算法在效率上有显著优势。虽然 FedEditor 并不是所有主动算法中最快的方案（如 Ram_lab 通过随机标签处理在速度上更有优势，而 FedEditor 需计算质心并构造正负样本对，耗时略长），但 FedEditor 作为主动遗忘方法，兼顾了效率与准确性：其训练速度远超被动遗忘算法，同时在剩余数据上的准确率不仅明显高于其他主动算法，还几乎与被动遗忘算法持平甚至略有超越。尤其在车联网等对安全性要求极高的场景下，为提升剩余数据的识别精度、哪怕多花费几秒训练时间也是完全值得的。

实验结果证明

遗忘质量

问题二：FedEditor的遗忘能力

TABLE III

COMPARISONS ON UNLEARNING EFFICIENCY, MODEL PREDICTIVE PERFORMANCE, AND UNLEARNING EFFECT ACROSS IMAGE DATASETS

Metrics	FedAvg	Re_train	Passive FU			Active FU					
			Eraser	Rap_train	Con_train	Ram_lab	GA	Ng_ewc	In_tea	PGD	Ours
FMNIST-LeNet											
Time(s)↓	991.85	730.92	139.03	668.16	14.76	3.44	3.77	39.63	**3.20**	3.93	5.79
Acc. on test data↑	91.22	91.32	85.37	87.62	91.26	80.46	70.04	78.26	73.42	82.07	**87.34**
Acc. on unlearned data↓	99.92	0.50	0.08	0.17	99.91	0.75	**0.00**	2.25	2.08	6.00	2.58
JSD to Re_train↓	0.0057	-	0.0001	0.0001	0.0058	**0.0001**	0.0005	0.0004	0.0003	0.0004	**0.0001**
AD to Re_train↓	1.3675	-	0.1914	0.1331	1.3673	**0.1311**	0.3605	0.3871	0.2945	0.3955	0.1665
ASR of MIA_metric→ 50.00	81.97	50.00	50.00	50.00	82.04	**50.00**	**50.00**	**50.00**	52.51	**50.00**	**50.00**
ASR of MIA_model→ 50.00	99.67	50.54	50.00	50.75	98.38	**49.96**	52.21	53.50	44.17	56.96	50.63
CIFAR10-ResNet18											
Time(s)↓	2089.48	1469.65	869.47	2471.21	31.48	8.83	7.78	181.87	19.04	**7.12**	19.80
Acc. on test data↑	66.64	68.90	55.69	68.34	67.08	54.28	63.78	64.03	59.35	54.54	**66.02**
Acc. on unlearned data↓	98.60	2.80	2.90	4.00	77.40	4.90	5.10	4.70	6.00	**3.60**	3.70
JSD to Re_train↓	0.0053	-	0.0009	0.0007	0.0031	0.0035	0.0013	0.0013	0.0041	**0.0011**	0.0012
AD to Re_train↓	1.3208	-	0.5298	0.4223	1.0602	0.9655	0.4519	**0.4453**	1.1530	0.5415	0.4806
ASR of MIA_metric→ 50.00	75.66	50.00	50.00	50.00	61.09	50.46	**50.00**	**50.00**	51.88	50.06	50.59
ASR of MIA_model→ 50.00	98.80	50.10	50.35	50.05	52.15	45.40	39.05	39.90	45.35	54.15	**48.95**
SVHN-ResNet18											
Time(s)↓	4161.02	2726.48	1054.01	4131.24	37.77	11.09	10.65	206.73	16.50	14.11	13.24
Acc. on test data↑	89.81	91.02	88.16	90.16	89.66	43.29	77.02	81.25	79.21	74.20	**88.66**
Acc. on unlearned data↓	96.93	1.02	1.36	1.50	7.58	9.56	**0.00**	0.61	1.02	2.11	3.14
JSD to Re_train↓	0.0045	-	0.0001	0.0001	0.0019	0.0035	0.0003	**0.0002**	0.0020	0.0007	**0.0002**
AD to Re_train↓	1.3465	-	0.1418	0.1091	0.7891	1.1602	0.1564	**0.1263**	0.8051	0.3894	0.1399
ASR of MIA_metric→ 50.00	58.28	50.00	50.00	50.00	50.07	**50.00**	**50.00**	**50.00**	**50.00**	**50.00**	50.02
ASR of MIA_model→ 50.00	96.07	50.34	51.57	49.93	48.87	49.69	**50.00**	50.44	49.52	50.17	49.69
CIFAR100-ResNet34											
Time(s)↓	3193.76	2220.50	868.36	3682.60	22.13	**4.74**	4.77	178.48	9.34	7.17	11.33
Acc. on test data↑	42.84	45.92	37.03	47.98	41.13	31.95	35.70	38.85	32.82	17.90	**45.90**
Acc. on unlearned data↓	98.70	3.30	1.90	3.40	76.10	4.00	0.60	**0.00**	5.60	44.30	0.80
JSD to Re_train↓	0.0052	-	0.0016	0.0014	0.0030	0.0031	0.0027	0.0026	0.0031	0.0036	**0.0021**
AD to Re_train↓	1.2841	-	0.6960	0.6507	1.0359	0.9273	0.7871	0.7553	0.9377	1.0244	**0.7048**
ASR of MIA_metric→ 50.00	84.66	50.00	50.00	50.00	68.49	**50.00**	**50.00**	**50.00**	53.49	63.97	51.16
ASR of MIA_model→ 50.00	97.60	50.00	50.10	49.55	48.75	49.00	52.15	52.20	48.80	51.20	**50.20**
GTSRB-ResNet18											
Time(s)↓	1366.17	781.99	523.14	1144.07	13.57	9.25	8.24	84.76	12.45	**5.48**	22.58
Acc. on test data↑	58.54	53.41	20.48	66.61	54.89	50.26	55.03	55.75	30.64	38.01	**57.51**
Acc. on unlearned data↓	97.93	5.07	3.75	10.71	98.12	**0.19**	4.13	1.69	34.21	2.63	2.25
JSD to Re_train↓	0.0062	-	0.0029	0.0005	0.0080	0.0050	0.0014	0.0017	0.0060	0.0027	**0.0006**
AD to Re_train↓	1.1797	-	0.7650	0.3814	1.1916	0.8309	0.4961	0.4780	0.8739	0.6910	**0.4629**
ASR of MIA_metric→ 50.00	84.32	50.00	50.00	50.00	90.85	52.98	**50.00**	**50.00**	75.15	**50.00**	50.08
ASR of MIA_model→ 50.00	99.53	50.28	49.71	48.21	93.14	49.53	53.28	55.26	49.71	50.84	**49.91**

TABLE IV

针对问题 2，即遗忘的有效性，从实验结果可以看出，FedEditor 在遗忘任务上表现十分出色。尽管与部分对比方法相比，遗忘识别率未必最低，但已经显著降低了模型对待遗忘数据的记忆，充分实现了预期的遗忘目标。同时，值得注意的是，遗忘识别率并非越低越好。例如，完全重训练的模型遗忘识别率也无法降到 0，而有些算法虽然实现了极低的遗忘识别率，却以牺牲剩余数据的准确性为代价，整体表现反而不佳。

这里需要重点解释一下论文中用来衡量“遗忘干净程度”的两个核心指标：JSD to Re_train↓ 和 AD to Re_train↓。这两个指标的关键点在于后缀的“to Re_train” —— 即和彻底删数据、从头完整重训（Retrain）的“黄金标准”模型进行对比。“↓”代表数值越小越好，也就是遗忘模型和重训模型的差距越小越好。

具体来说：

1. JSD (Jensen-Shannon Divergence，詹森 - 香农散度)：这个指标用来衡量概率分布之间的相似度。在本场景下，就是比较你的遗忘模型与重训模型，在同一个测试样本上输出的预测概率分布有多相像。JSD 值越接近 0，说明你的遗忘模型在输出层上已经非常接近于“完美遗忘”的重训模型，没有残留异常行为，遗忘够“干净”。
2. AD (Activation Distance / Absolute Distance，激活距离/绝对距离)：这个指标反映两个模型在处理相同输入时，其内部特征表示或输出向量之间的空间距离。如果说 JSD 比的是“猜的结果”，AD 看

的则是底层“思考过程”上的接近程度。AD 越低，表示遗忘模型在神经元、特征层参数等深层面也已和重训模型非常接近，遗忘的效果非常彻底。

论文实验展示，FedEditor 在这两个核心指标（JSD to Re_train↓ 和 AD to Re_train↓）上的表现整体优于基准模型，说明其遗忘算法不仅实现了高效“删除”，而且最小化了与重训的差距，在不影响剩余数据性能的前提下，遗忘结果足够彻底和“纯净”。

实验结果证明

遗忘质量

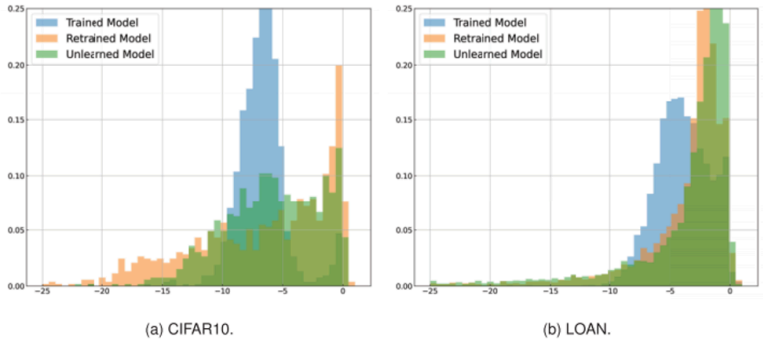


Fig. 6. Distributions of the entropy of models' output on $\mathcal{D}_f$.

熵分布图

分类标签图

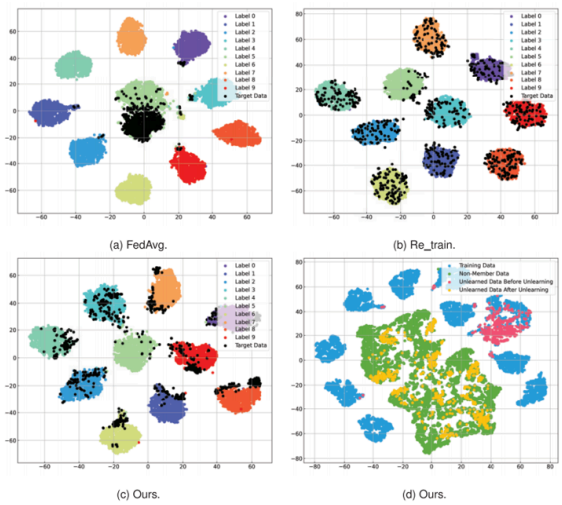


Fig. 7. t-SNE visualizations of representations from models.

身份图

从熵分布图可以直观地看出，FedEditor 有效地模拟了重训练效果：蓝色表示原始模型，其熵分布集中且数值较低，而重训练模型和遗忘模型的熵分布整体更高、更分散，并且二者在分布区域上高度重叠，显示出遗忘模型成功逼近了理想的重训练状态。进一步，通过特征表示的可视化结果也能明显看出模型调整后的作用：图 a 展示的是基准模型，不同类别的数据在特征空间中聚成明显的团块，类别间区分清晰；图 c 则表明在彻底重训练后，待遗忘数据的原本聚类被有效打散，分布渗透到其他类别的团块之中，模型已无法准确区分这些待遗忘样本的原始类别。图 d 进一步展示了特征分布的动态变化：蓝色为成员数据，绿色为非成员数据，粉色代表待遗忘数据的初始状态，黄色为遗忘后状态。可见，遗忘后待遗忘数据的特征向量由原先的高度聚集状态被有意识地驱散，分布到非成员数据周围，与作者提出的设计目标高度一致。

问题三：通过FedEditor保持全局模型预测性能的稳定性

提取文字　更多功能

实验结果证明

预测性能

TABLE III

COMPARISONS ON UNLEARNING EFFICIENCY, MODEL PREDICTIVE PERFORMANCE, AND UNLEARNING EFFECT ACROSS IMAGE DATASETS

Metrics	FedAvg	Re_train	Passive FU			Active FU					
			Eraser	Rap_train	Con_train	Ram_lab	GA	Ng_ewc	In_tea	PGD	Ours
FMNIST-LeNet											
Time(s)↓	991.85	730.92	139.03	668.16	14.76	3.44	3.77	39.63	**3.20**	3.93	5.79
Acc. on test data↑	91.22	91.32	85.37	87.62	91.26	80.46	70.04	78.26	73.42	82.07	**87.34**
Acc. on unlearned data↓	99.92	0.50	0.08	0.17	99.91	0.75	**0.00**	2.25	2.08	6.00	2.58
JSD to Re_train↓	0.0057	-	0.0001	0.0001	0.0058	**0.0001**	0.0005	0.0004	0.0003	0.0004	**0.0001**
AD to Re_train↓	1.3675	-	0.1914	0.1331	1.3673	**0.1311**	0.3605	0.3871	0.2945	0.3955	0.1665
ASR of MIA_metric→ 50.00	81.97	50.00	50.00	50.00	82.04	**50.00**	**50.00**	**50.00**	52.51	**50.00**	**50.00**
ASR of MIA_model→ 50.00	99.67	50.54	50.00	50.75	98.38	**49.96**	52.21	53.50	44.17	56.96	50.63
CIFAR10-ResNet18											
Time(s)↓	2089.48	1469.65	869.47	2471.21	31.48	8.83	7.78	181.87	19.04	**7.12**	19.80
Acc. on test data↑	66.64	68.90	55.69	68.34	67.08	54.28	63.78	64.03	59.35	54.54	**66.02**
Acc. on unlearned data↓	98.60	2.80	2.90	4.00	77.40	4.90	5.10	4.70	6.00	**3.60**	3.70
JSD to Re_train↓	0.0053	-	0.0009	0.0007	0.0031	0.0035	0.0013	0.0013	0.0041	**0.0011**	0.0012
AD to Re_train↓	1.3208	-	0.5298	0.4223	1.0602	0.9655	0.4519	**0.4453**	1.1530	0.5415	0.4806
ASR of MIA_metric→ 50.00	75.66	50.00	50.00	50.00	61.09	50.46	**50.00**	**50.00**	51.88	50.06	50.59
ASR of MIA_model→ 50.00	98.80	50.10	50.35	50.05	52.15	45.40	39.05	39.90	45.35	54.15	**48.95**
SVHN-ResNet18											
Time(s)↓	4161.02	2726.48	1054.01	4131.24	37.77	11.09	**10.65**	206.73	16.50	14.11	13.24
Acc. on test data↑	89.81	91.02	88.16	90.16	89.66	43.29	77.02	81.25	79.21	74.20	**88.66**
Acc. on unlearned data↓	96.93	1.02	1.36	1.50	7.58	9.56	**0.00**	0.61	1.02	2.11	3.14
JSD to Re_train↓	0.0045	-	0.0001	0.0001	0.0019	0.0035	0.0003	**0.0002**	0.0020	0.0007	**0.0002**
AD to Re_train↓	1.3465	-	0.1418	0.1091	0.7891	1.1602	0.1564	**0.1263**	0.8051	0.3894	0.1399
ASR of MIA_metric→ 50.00	58.28	50.00	50.00	50.00	50.07	**50.00**	**50.00**	**50.00**	**50.00**	**50.00**	50.02
ASR of MIA_model→ 50.00	96.07	50.34	51.57	49.93	48.87	49.69	**50.00**	50.44	49.52	50.17	49.69
CIFAR100-ResNet34											
Time(s)↓	3193.76	2220.50	868.36	3682.60	22.13	**4.74**	4.77	178.48	9.34	7.17	11.33
Acc. on test data↑	42.84	45.92	37.03	47.98	41.13	31.95	35.70	38.85	32.82	17.90	**45.90**
Acc. on unlearned data↓	98.70	3.30	1.90	3.40	76.10	4.00	0.60	**0.00**	5.60	44.30	0.80
JSD to Re_train↓	0.0052	-	0.0016	0.0014	0.0030	0.0031	0.0027	0.0026	0.0031	0.0036	**0.0021**
AD to Re_train↓	1.2841	-	0.6960	0.6507	1.0359	0.9273	0.7871	0.7553	0.9377	1.0244	**0.7048**
ASR of MIA_metric→ 50.00	84.66	50.00	50.00	50.00	68.49	**50.00**	**50.00**	**50.00**	53.49	63.97	51.16
ASR of MIA_model→ 50.00	97.60	50.00	50.10	49.55	48.75	49.00	52.15	52.20	48.80	51.20	**50.20**
GTSRB-ResNet18											
Time(s)↓	1366.17	781.99	523.14	1144.07	13.57	9.25	8.24	84.76	12.45	**5.48**	22.58
Acc. on test data↑	58.54	53.41	20.48	66.61	54.89	50.26	55.03	55.75	30.64	38.01	**57.51**
Acc. on unlearned data↓	97.93	5.07	3.75	10.71	98.12	**0.19**	4.13	1.69	34.21	2.63	2.25
JSD to Re_train↓	0.0062	-	0.0029	0.0005	0.0080	0.0050	0.0014	0.0017	0.0060	0.0027	**0.0006**
AD to Re_train↓	1.1797	-	0.7650	0.3814	1.1916	0.8309	0.4961	0.4780	0.8739	0.6910	**0.4629**
ASR of MIA_metric→ 50.00	84.32	50.00	50.00	50.00	90.85	52.98	**50.00**	**50.00**	75.15	**50.00**	50.08
ASR of MIA_model→ 50.00	99.53	50.28	49.71	48.21	93.14	49.53	53.28	55.26	49.71	50.84	**49.91**

关于第三个问题预测能力的准确性，可以看到图中 FedEditor 的数据非常亮眼，在各个算法中都名列前茅，这因为 FedEditor 有明确的对齐目标，而且专注于表示层的修改，相比之下，Rap_train 等被动方法虽然准确但太慢，而 GA 等主动方法虽然快但会严重破坏模型性能。

$$Total\ Loss = \underbrace{\alpha \cdot l_u}_{\text{往错误方向拉}} + \underbrace{\beta \cdot l_r}_{\text{保持正确数据准确}} + \underbrace{\gamma \cdot l_d}_{\text{不让参数跑太远}}$$

l_u：知识遗忘损失（Knowledge Unlearning Loss）

l_r：记忆保留损失（Memory Preservation Loss）

l_d：偏差抵抗损失（Deviation Resistance Loss）

实验结果证明

组件消融实验

TABLE V

ABLATION STUDY ON DIFFERENT COMPONENTS

Components	Metrics		Components	Metrics	
	Acc. on D_f ↓	Acc. on D_t ↑		Acc. on D_f ↓	Acc. on D_t ↑
CIFAR10-ResNet18			GTSRB-ResNet18		
FedAvg	98.60	66.64	FedAvg	97.93	58.54
ℓ_u	6.70	65.51	ℓ_u	1.88	57.22
ℓ_r	45.20	65.96	ℓ_r	95.68	55.15
ℓ_d	75.40	65.92	ℓ_d	97.37	55.57
$\ell_u + \ell_r$	3.70	65.92	$\ell_u + \ell_r$	2.26	57.49
$\ell_u + \ell_d$	6.80	65.51	$\ell_u + \ell_d$	1.88	57.24
$\ell_r + \ell_d$	46.50	65.99	$\ell_r + \ell_d$	95.68	55.20
$\alpha = 1.0$	43.80	66.06	$\alpha = 1.0$	90.60	54.65
$\gamma = 1.0$	53.90	66.18	$\gamma = 1.0$	12.41	57.53
Ours	3.70	66.02	Ours	2.25	57.51

全局损失函数由三部分组成，通过消融实验证明三部分缺一不可，且 FedEditor 精心选择 a 和 r 最后取得了平衡点

为什么不涉及β，在多目标优化中，通常需要选定一个基准。由于 lr 代表的是“保持正常功能”这个最基础的目标，作者把它设为 1（基准单位）。默认 a 是 10,r 是 0.1

问题四：FedEditor的适用性

实验结果证明

不同分布的非成员数据对训练数据的影响

TABLE X　　非成员数据集质量越来越差

TABLE NON-MEMBER DATA WITH DIFFERENT DISTRIBUTIONS FROM THE TRAINING DATA

Metrics	FedAvg	Re_train	Case 1	Case 2	Case 3	Case 4	Case 5
Acc. on D_t ↑	66.64	68.90	66.02	65.64	65.81	65.50	65.66
Acc. on D_f ↓	98.60	2.80	3.70	3.80	3.20	3.50	3.30
JSD to Re_train	0.0053	-	0.0012	0.0013	0.0014	0.0013	0.0012
AD to Re_train	1.3208	-	0.4806	0.4984	0.5116	0.4964	0.4884
ASR of MIA_metric	75.66	50.00	50.59	50.29	50.40	50.30	50.30
ASR of MIA_model	98.80	50.10	48.95	48.80	49.45	48.75	48.50

对计算错误类别质心的图质量要求不高

不同分布的非成员数据对模型训练的影响：图中从左到右，所选非成员数据集的质量依次降低，但实验结果显示，无论非成员数据质量如何，模型的遗忘效果与剩余数据的准确率基本保持稳定。这表明，甚至随意选取一些无关图片作为非成员数据，也能够有效支撑算法实现遗忘目标。

实验结果证明

FedEditor产生的额外开销

TABLE XI

THE COMPUTATIONAL COSTS OF REPRESENTATION EXTRACTION AND RETRIEVING CENTROIDS ACROSS VARIOUS DATASETS

Data	FMNIST	CIFAR10	SVHN	CIFAR100	GTSRB	LOAN	NBAIOT
Instances	10000	10000	26032	10000	12630	90430	110000
Classes	10	10	10	100	43	9	11
Time$_1$(s)	0.95	3.21	4.03	1.81	5.27	8.38	10.3
Time$_2$(s)	0.0018	0.0017	0.0017	0.0168	0.0076	0.0020	0.0019

模型训练时间

算法消耗时间

TABLE XII

THE COMPUTATIONAL COSTS OF REPRESENTATION EXTRACTION WITH DIFFERENT PROPORTIONS OF NON-MEMBER DATA

Metrics	FedAvg	Re_train	20%	40%	60%	80%	100%
Time$_1$(s)	-	-	0.51	1.25	1.34	2.21	3.21
Acc. on D_t ↑	66.64	68.90	65.78	65.75	65.80	66.02	66.02
Acc. on D_f ↓	98.60	2.80	3.80	4.00	3.90	4.20	3.70
JSD to Re_train	0.0053	-	0.0012	0.0012	0.0012	0.0012	0.0012
AD to Re_train	1.3208	-	0.4804	0.4833	0.4783	0.4828	0.4806
ASR of MIA_metric	75.66	50.00	50.59	50.58	50.68	50.65	50.59
ASR of MIA_model	98.80	50.10	49.00	49.15	49.15	49.15	48.95

非成员数据使用 比例对结果影响不大

FedEditor 带来的额外计算和通信开销极小。实验还表明，即使显著降低非成员数据的比例，对最终模型性能和遗忘效果的影响也非常有限，算法鲁棒性较强，无需高比例非成员数据即可达到理想表现。

实验结果证明

忘掉一整类事物

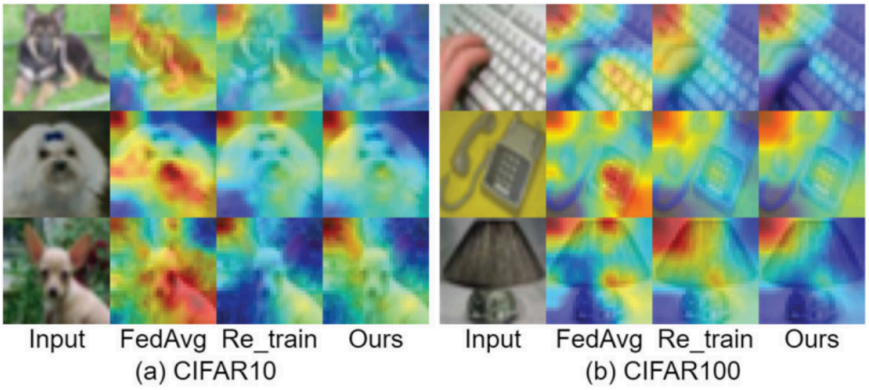


Fig. 9. Attention map on class unlearning tasks.

提取文字 ••• 更多功能

作者还进一步通过实验探究了能否实现对整类对象的遗忘（如遗忘所有猫或所有狗）。结果显示，在原始模型中，模型的注意力主要集中在动物的眼睛、鼻子等关键区域，以辅助类别判别；而理想模型则不再关注这些区域。经过 FedEditor 算法处理后，模型的注意力分布与理想重训练模型高度一致，充分证明了该方法在整类遗忘场景下同样具备良好效果和可行性。

实验结果证明

其他测试

申请遗忘的目标车辆数量影响

TABLE VI

IMPACT OF VARYING NUMBERS OF TARGET VEHICLES

Metrics	Scheme	2	4	6	8	10
Acc. on $D_t \uparrow$	FedAvg	66.64	66.19	64.74	66.52	63.53
	Re_train	68.90	67.58	66.40	67.78	66.25
	Ours	66.02	65.34	63.90	65.27	62.47
Acc. on $D_f \downarrow$	FedAvg	98.60	98.80	99.17	99.02	99.62
	Re_train	2.80	3.70	4.53	4.47	5.44
	Ours	3.70	4.90	6.87	4.37	0.46
JSD to Re_train	FedAvg	0.0053	0.0028	0.0019	0.0014	0.0014
	Re_train	-	-	-	-	-
	Ours	0.0012	0.0006	0.0004	0.0002	0.0002
AD to Re_train	FedAvg	1.3208	1.3137	1.3055	1.3020	1.2938
	Re_train	-	-	-	-	-
	Ours	0.4806	0.4823	0.4709	0.4452	0.4566
ASR of MIA_metric	FedAvg	75.66	76.91	78.54	76.20	84.33
	Re_train	50.00	50.00	50.00	50.00	50.00
	Ours	50.59	50.01	50.01	50.00	50.02
ASR of MIA_model	FedAvg	98.80	99.07	98.80	98.71	98.89
	Re_train	50.10	50.02	49.73	50.15	50.80
	Ours	48.95	49.03	48.53	50.20	49.18

遗忘数量影响

TABLE VII

IMPACT OF VARYING PROPORTIONS OF UNLEARNED DATA

Metrics	Scheme	10%	15%	20%	25%	30%
Acc. on $D_t \uparrow$	FedAvg	66.64	66.85	66.53	66.52	66.33
	Re_train	68.90	67.73	67.59	67.47	68.98
	Ours	66.02	65.24	65.46	65.32	64.98
Acc. on $D_f \downarrow$	FedAvg	98.60	98.53	98.35	98.36	98.37
	Re_train	2.80	4.47	4.60	3.52	3.87
	Ours	3.70	2.13	2.00	2.24	1.47
JSD to Re_train	FedAvg	0.0053	0.0033	0.0022	0.0017	0.0014
	Re_train	-	-	-	-	-
	Ours	0.0012	0.0008	0.0006	0.0004	0.0004
AD to Re_train	FedAvg	1.3208	1.3006	1.2981	1.3082	1.3060
	Re_train	-	-	-	-	-
	Ours	0.4806	0.5017	0.4768	0.4379	0.4400
ASR of MIA_metric	FedAvg	75.66	75.16	74.77	74.67	74.57
	Re_train	50.00	50.00	50.00	50.00	50.00
	Ours	50.59	50.08	50.13	50.15	50.03
ASR of MIA_model	FedAvg	98.80	98.63	98.73	98.64	98.93
	Re_train	50.10	49.66	50.07	49.84	50.15
	Ours	48.95	50.13	50.85	50.70	50.65

参与实验的车辆数量影响

TABLE VIII

IMPACT OF VARYING NUMBERS OF PARTICIPATING VEHICLES

Metrics	Scheme	10	20	30	40	50
Acc. on $D_t \uparrow$	FedAvg	66.64	61.93	53.15	56.40	54.01
	Re_train	68.90	64.46	53.41	57.27	54.91
	Ours	66.02	60.19	51.69	55.00	52.01
Acc. on $D_f \downarrow$	FedAvg	98.60	96.60	95.68	95.70	94.60
	Re_train	2.80	5.20	5.12	4.10	5.90
	Ours	3.70	1.20	7.55	4.30	5.70
JSD to Re_train	FedAvg	0.0053	0.0047	0.0037	0.0035	0.0022
	Re_train	-	-	-	-	-
	Ours	0.0012	0.0011	0.0013	0.0009	0.0009
AD to Re_train	FedAvg	1.3208	1.2669	1.2250	1.2254	1.1437
	Re_train	-	-	-	-	-
	Ours	0.4806	0.5187	0.6984	0.5807	0.6584
ASR of MIA_metric	FedAvg	75.66	74.56	75.38	70.85	71.04
	Re_train	50.00	50.00	50.00	50.00	50.00
	Ours	50.59	51.46	55.27	50.18	52.33
ASR of MIA_model	FedAvg	98.80	97.75	96.98	96.70	96.50
	Re_train	50.10	49.60	49.94	50.15	51.20
	Ours	48.95	51.10	50.95	52.65	50.75

数据不均匀对结果的影响

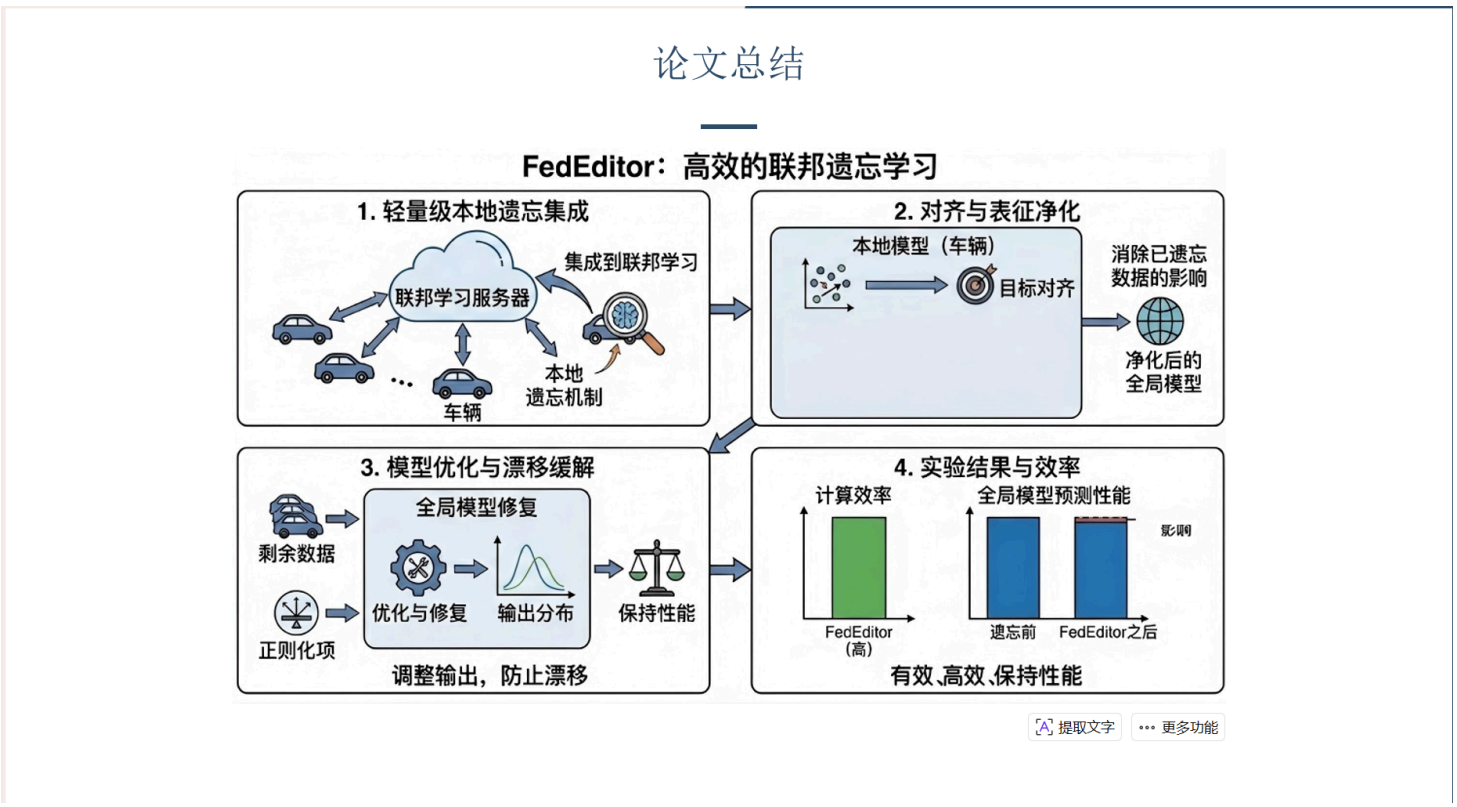
TABLE IX

IMPACT OF DATA HETEROGENEITY AMONG VEHICLES

Metrics	FedAvg	Re_train	Ours	FedAvg	Re_train	Ours
	$\mu = 1.0$			$\mu = 10.0$		
Acc. on $D_t \uparrow$	61.26	68.22	58.28	70.98	71.27	67.71
Acc. on $D_f \downarrow$	96.82	0.60	4.29	92.00	0.70	1.56
JSD to Re_train	0.0053	-	0.0024	0.0053	-	0.0011
AD to Re_train	1.3256	-	0.9913	1.2864	-	0.5351
ASR of MIA_metric	75.63	50.00	53.48	67.01	50.00	50.94
ASR of MIA_model	97.63	49.69	49.09	98.07	49.79	51.97

最后，作者还在多种车联网现实复杂场景下对 FedEditor 的稳定性进行了深入验证。实验涵盖了不同类别和任务设置，即使在各种极端或高难度的环境中，FedEditor 依然展现出显著的稳健性。尽管任务复杂度提升会带来模型性能的轻微下降，但整体表现依然优异，各项指标变化均处于可接受范围之内，充分证明了算法的通用性与稳定性。

6. 总结与思考



FedEditor 创新性地将轻量级本地遗忘机制集成到联邦学习流程中。针对待遗忘车辆的数据，FedEditor 精确构建了目标对齐方案，并结合表征空间净化技术，有效消除了遗忘数据对全局模型的影响。同时，FedEditor 利用目标车辆的剩余数据优化模型输出分布，通过引入正则化项抑制模型漂移，确保在遗忘过程中全局模型的预测能力稳定。大量实验结果进一步验证，FedEditor 能以高效率完成有效遗忘，同时几乎不影响全局模型对遗忘数据以外样本的预测性能。

7. 参考文献

原文引用格式（IEEE 格式）：

```
X. Yuan, J. Liu, B. Wang, G. Chen, X. Xu, J. Wang, T. Li, and W. Wang,
"FedEditor: Efficient and Effective Federated Unlearning in Cooperative
Intelligent Transportation Systems," IEEE Transactions on Information
Forensics and Security, vol. 20, pp. 6560-6575, 2025.
DOI: 10.1109/TIFS.2025.3583231
```

相关链接：

- 论文 DOI: https://doi.org/10.1109/TIFS.2025.3583231
- 代码仓库：https://github.com/XXiaoY/Fededitor